

Network Security

Lab 04 - Network Access Control – User Authentication

1. Learning Objectives

- What is Kerberos?
- What is principal?
- What is Ticket Granting Ticket?
- What is Service Ticket?
- What is Kerberized Service?

2. Equipment/software

Each team should have access to

- VMware Workstation Pro on Linux/Windows or Fusion Pro on OSX

3. Best Practices (Trouble Shooting is How You Learn Networking!)

4. Exercises

MAKE SURE TO PROVIDE NETWORK DIAGRAMs AND TABLEs CONTAINING INFORMATION ABOUT THE MACHINES INVOLVED (SUCH AS, IP ADDRESSES, MAC ADDRESSES, ETC.)

4.1 Exercise 1: Kerberos (Authentication via Tickets)

Introduction

Here are a few terms that are good to understand before setting up a Kerberos server.

- **Principal:** any users, computers, and services provided by servers need to be defined as Kerberos Principals.
- **Instances:** are used for service principals and special administrative principals.
- **Realms:** the unique realm of control provided by the Kerberos installation. Think of it as the domain or group your hosts and users belong to. Convention dictates the realm should be in uppercase. By default, Ubuntu will use the DNS domain converted to uppercase (EXAMPLE.COM) as the realm.
- **Key Distribution Center:** (KDC) consist of three parts, a database of all principals, the authentication server, and the ticket granting server. For each realm there must be at least one KDC.
- **Ticket Granting Ticket:** issued by the Authentication Server (AS), the Ticket Granting Ticket (TGT) is encrypted in the user's password which is known only to the user and the KDC.
- **Ticket Granting Server:** (TGS) issues service tickets to clients upon request.
- **Tickets:** confirm the identity of the two principals. One principal being a user and the other a service requested by the user. Tickets establish an encryption key used for secure communication during the authenticated session.
- **Keytab Files:** are files extracted from the KDC principal database and contain the encryption key for a service or host.

To put the pieces together, a Realm has at least one KDC, preferably more for redundancy, which contains a database of Principals. When a user (principal) logs into a workstation that is

configured for Kerberos authentication, the KDC issues a Ticket Granting Ticket (TGT). If the user supplied credentials match, the user is authenticated and can then request tickets for Kerberized services from the Ticket Granting Server (TGS). The service tickets allow the user to authenticate to the service without entering another username and password.

Step 1:

Preparation

T1. You need 3 VMs: Ubuntu Server VM1 (client)/NAT, Ubuntu Server VM2 (service)/NAT and Ubuntu Server VM3 (kdcserver)/NAT. Make sure to execute `sudo apt update && upgrade` on each VM. See Figure E2-1 for the exercise network arrangement.

Navigate to the Virtual Network Editor on VMware, for `vmnet8`, enter subnet IP range is `192.168.1.0/24`, subnet mask `255.255.255.0`, **uncheck** DHCP, and **check in** the virtual interface on the Host. Change the hostnames, and update the `/etc/hosts` file according to Table E2-1.

Table E2-1 - VMs hostnames and records in `/etc/hosts`

	<code>/etc/hostname</code>	<code>/etc/hosts</code> (replace 127.0.1.1 ubuntu with)
VM1	<code>client.kerberos.com</code>	<code>192.168.1.10 client.kerberos.com client</code>
VM2	<code>service.kerberos.com</code>	<code>192.168.1.20 service.kerberos.com service</code>
VM3	<code>kdcserver.kerberos.com</code>	<code>192.168.1.30 kdcserver.kerberos.com kdcserver</code>

On modern Ubuntu (22.04/24.04/24.04.1), the **default** is that `/etc/resolv.conf` is a symlink to `/run/systemd/resolve/stub-resolv.conf`, which in turn may point at `/run/systemd/resolve/resolv.conf` depending on how resolved is configured.

Perform the following commands on each of the 3 VMs to ensure that the operating system point to the custom DNS server to be created later:

```
sudo rm -f /etc/resolv.conf
sudo ln -s ../run/systemd/resolve/stub-resolv.conf /etc/resolv.conf
sudo reboot
```

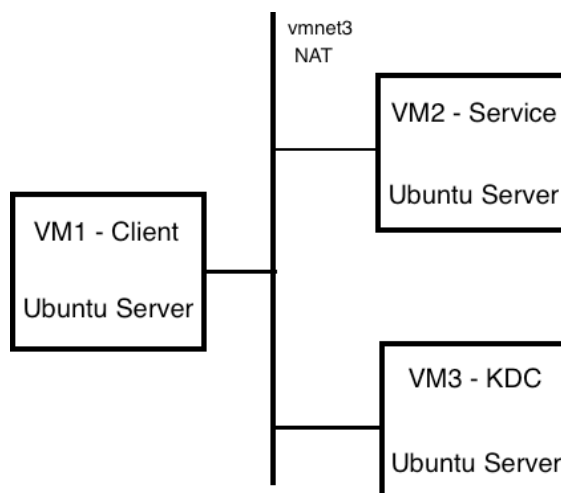


Figure E2-1: Exercise Network Arrangement

Configure static IP addresses on the the Ethernet interfaces of the 3 VMs. Appendix A shows an example of VM1 configuration using netplan. Check name resolution addresses using
resolvectl status | grep "DNS Servers" -A2

It is critical that you understand the DNS resolution process. If a name needs to be resolved, VM1 operating system name resolution process would look up the hosts file, if the name is not resolved to an address, then VM1, for example, would look up the systemd-resolve file to find the next name server (or a gateway to a nameserver) such as 192.168.1.30 in our case, if that does not work, then the next available nameserver IP in the, in our case it is 192.168.1.2, if that does not work, then the next available nameserver IP in the 8.8.8.8, if that does not work then the look up fails.

Install and configure a local DNS server on VM3

T2. Before you start the local DNS installation, let's find out what transport ports are up on VM3 by executing:

```
ss -nlu  
ss -nlut
```

Keep a record of the results to use shortly for comparison. Now, let's install and configure a local DNS server on the kdcserver.kerberos.com VM3. Next, let's install the package called bind9 on VM3.

```
sudo apt install bind9 bind9-utils
```

Note: In large-scale networks, the local DNS server may be running on a machine of its own.

Next, you need to configure /etc/bind/named.conf.local file according to Appendix B. This file points to two other files: kerberos.com.zone and kerberos.com.rev. The first has the local DNS zone information required for name to ip address resolution and the second contains the records for reverse lookup. Create and configure the zone and rev files according to Appendix B.

Restart the bind9 local DNS Server.

```
sudo systemctl restart bind9.service
```

Conduct the tests in Appendix C. Make sure that they are all successful!

Execute **ss -nlu & ss -lu** to verify that the DNS local server related transport ports are up and running. You should have 4 IPv4 additional transport ports **netstat** (or **ss**) records relevant to the local DNS server.

- Q1 – List the 4 transport ports records that are relevant to local DNS server and explain what each of them represents. Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources

If our local DNS is successfully installed, configured and restarted, then the following list of additional tests should be successful and without errors. On VM3, check connectivity with and the name resolution of VM1 and VM2.

```
ping client.kerberos.com
```

```
ping service.kerberos.com
```

```
ping www.google.com #it may take few seconds to star for the first time!
```

You should repeat the above tests on VM1 (check connectivity with and name resolution of VM2 and VM3) and on VM2 (check connectivity with and name resolution of VM1 and VM3).

- Q2 – Explain how the VMs are able to resolve the `www.google.com` name although you have no records for `www.google.com` in the local DNS server zone file. Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources

Install and configure a local ntp server on VM3

T3. Next, you need to install a local ntp server on all three VMs by executing the following command on each of them:

```
sudo apt install ntp ntpdate
```

Test the ntp servers (in terms of how far off their date and time stamps are) on each virtual machine using the following commands:

```
ntpq -p # this may take a while
```

```
ntpdate -dv 192.168.1.<n> # this may take a while
```

where n is 10 on VM1, 20 on VM2 and 30 on VM3. If you encounter a problem with the `ntpdate -dv`, restart the ntp server using **sudo systemctl restart ntp.service**.

- Q3 –
 - (a) What is the transport port number of the ntp server?
 - (b) Explain what the `ntpq -p` and `ntpdate -dv <ip address>` do.
 - (c) Identify the VMs ntp time stamps and whether their differences are acceptable for the successful operation of Kerberos protocol/system.

Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources

Step 2: Install and configure Kerberos on VM3 server

Set up Kerberos Realm

T1. On VM3 (`kdcserver.kerberos.com`), you need to install two packages: `krb5-kdc` and `krb5-admin-server` by executing:

```
sudo apt install krb5-kdc krb5-admin-server
```

During the installation, you should be asked the following questions (answers are shown in the square brackets)

- Default Kerberos version 5 realm? [**KERBEROS.COM**]
- Kerberos servers for your realm? [**kdcserver.kerberos.com**]
- Administrative server for your realm? [**kdcserver.kerberos.com**]

The installation will complete successfully. However, the start of the kdc server may fail. That should be fixed shortly. Now, you need to create our Kerberos realm by executing the following command:

```
sudo krb5_newrealm
```

When you are asked, enter **hellokdc** as the KDC database master key. This may take a while.

Now, restart and check the status of the kdc server using the following commands:

```
sudo systemctl restart krb5-kdc.service
```

sudo systemctl status krb5-kdc.service

These should be successful.

Save the content of /etc/krb5.conf into a file. Next, you need to become the root user by typing **sudo su**. Save the content of /etc/krb5kdc/kdc.conf into a file. Exit the root account. These will be very useful in troubleshooting later.

T2. Next, you need to edit the etc/krb5.conf by modifying, entering and/or leaving unchanged the following lines as necessary:

```
[libdefaults]
    default_realm = KERBEROS.COM                #most likely it is already there!
...
[realms]
    KERBEROS.COM = {
        kdc = kdcserver.kerberos.com           #most likely it is already there!
        admin_server = kdcserver.kerberos.com  #most likely it is already there!
    }
...
[domain_realm]
...
    kerberos.com = KERBEROS.COM                #added
    .kerberos.com = KERBEROS.COM               #added
```

Next, you need to uncomment or enter `*/admin *` line in /etc/krb5kdc/kadm5.acl file. You need to be root (using sudo su) to have access to kadm.acl file. Once you're done, exit root.

Now, restart the krb5kdc (krb5-kdc) and kadmin (krb5-admin-server) processes/services.

```
sudo systemctl restart krb5-kdc.service
sudo systemctl status krb5-kdc.service
sudo systemctl restart krb5-admin-server.service
sudo systemctl status krb5-admin-server.service
```

These should be successful. Now, let's check the transport ports that are up by executing the following:

```
netstat -nltn
netstat -ltu
ps -aux | grep krb5*
ps -aux | grep kadm*
sudo lsof -Pan -p <pid> -i # one for krb5 related pid and one for kadm related pid.
```

Confirm that krb5kdc and kadmin are up and running. If these processes are not running, then you need to start or restart them and troubleshoot as necessary.

- Q1 –
 - (a) Identify the IPv4 transport port(s) associated with the krb5kdc server and
 - (b) those that are associated with the krb5-admin-server (tcp and udp based).
 - (c) Describe the service(s) offered by each transport port. Hint: 6 (3 tcp, 3 udp)
- Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources

Also, check the content of /etc/krb5kdc and /var/lib/krb5kdc directories (you need sudo su). You should see the following files in the /etc/krb5kdc directory **kadm5.acl kdc.conf stash** and the following files **principal principal.kadm5 principal.kadm5.lock principal.ok** in the /var/lib/krb5kdc directory. Exit root.

- Q2 – Describe briefly what these files are for and how they are used by the Kerberos platform. Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources

Add users (principals)

T3. Now, it's time to add users (principals). You're to use the kadmin.local utility on VM3 to do so. First, type

```
sudo kadmin.local
```

Find out the available commands under kadmin.local by typing a question mark after the kadmin.local prompt.

```
kadmin.local: ?
```

When you are prompted with "kadmin.local:", type (only the bold string)

```
kadmin.local: listprincs
```

- Q3 – Provide a copy of the principals already established, by default, on the Kerberos server. Describe briefly what each represents. Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources

Next, you need to type (only the bold text):

```
kadmin.local: addprinc root
```

Enter **helloroot** as the password for the root user (principal). Add another user/principal such as checkout as follows:

```
kadmin.local: addprinc checkout
```

Enter **checkout** as the password for the checkout user (principal). You can verify that the two users are successfully created by typing the following at the kadmin.local prompt.

```
kadmin.local: listprincs
```

You should see two additional principals: root and checkout (in addition to the original default principals). Save the results. Now exit kadmin.local by typing q and hit return/enter.

Local TGT creation (equivalent to first and only login - SSO)

T4. Launch Wireshark (using ssh -X forwarding remote login) on VM3, setup Wireshark to capture on the lo interface and filter on kerberos. Start capturing.

On VM3 (for testing purposes), check that there are no TGTs (Ticket Granting) Tickets for root by typing the following:

```
sudo klist
```

The response should indicate that there is none. Create a TGT (Ticket Granting) Ticket for root by typing the following:

sudo kinit -V root

Enter the password for root. If you receive the following response “Authenticated to Kerberos v5”, then you are in business. You should see at least 4 krb5 packets on Wireshark. Note that the **kinit** command allows us to obtain or renew a Kerberos ticket-granting ticket for a given principal. Let’s check the validity of the TGT by type:

sudo klist

Stop packet capturing on Wireshark and export the Kerberos packets to **TGT-root-local-VM3-E2S2T4-lo-VM3-xforwarding-on-HOST.pcapng**

- Q4 –
 - (a) Describe in details the messages exchanged and captured on Wireshark (lo interface) for the locally initiated request for a TGT. Make use of the klist response and the exported Kerberos packets. Hint: timestamp encrypted with a key derived from the users’ password hash.
 - (b) Identify the principle who requested the TGT.
Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources
- Q5 –
 - (a) In the last response packet (AS-REP), identify the data represents the ticket and the session key data.
 - (b) Verify that the session key is encrypted by a key derived from the user’s password hash.
Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources

Step 3: Configure Kerberos for a Client and a Service (SSO for SSH Service)

On VM1 (client.kerberos.com)

T1. install krb5-user

sudo apt install krb5-user

You should be presented with the same questions you were asked during the krb5-kdc server installation. The answers are the same: KERBEROS.COM, kdcserver.kerberos.com and kdcserver.kerberos.com.

Check that you have /etc/krb5.conf. Edit /etc/krb5.conf to be identical to what you have for the kdcserver in step 2. Make sure that you have connectivity from VM1 to VM3 and VM2.

Now that you have a client ready to use Kerberos, the first order of business is for the client to request a TGT (first login). First, launch Wireshark on VM1 (x forwarding from HOST), configure to capture packets on the interface connected to vmnet8 (you need to use the Ethernet interface name on VM1), and apply kerberos as a filter.

Let’s check if you have any left-over tickets by executing the following command:

sudo klist

If there is any, you need to destroy them by applying the following command:

sudo kdestroy

To request a Ticket Granting ticket for user root, you need to execute the following command:

sudo kinit -V root

root is one of two users (principals) you created earlier.

Enter the password you configured earlier (helloroot). If the TGT is created successfully, you should receive a response similar to the one shown in Figure E2-2.

```
checkout@client:~$ sudo kinit -V root
Using default cache: /tmp/krb5cc_0
Using principal: root@KERBEROS.COM
Password for root@KERBEROS.COM:
Authenticated to Kerberos v5
```

Figure E2-2 – Successful creation and delivery of TGT for user root

- T2.** Let's check that the TGT is actually created and find out its properties by executing the following command:

sudo klist

You should be presented with a detailed account of the TGT.

Stop Wireshark and export the Kerberos captured packets into **TGT-root-remote-VM1-E2S3T2-to-VM3-xforwarding-on-HOST.pcapng**.

Q1 - Explain in details the Kerberos packets exchanged between VM1 and VM3 in the process of creating a TGT. Hint: Make extensive use of the Wireshark captured packets! Make sure to use a screenshot of the relevant packets captured by Wireshark! Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources

Next, you need to configure the SSH client to use Kerberos on the client machine (VM1). Uncomment the following lines in the /etc/ssh/sshd_config file. Also, change the "no" in the first line to "yes".

GSSAPIAuthentication yes
GSSAPICleanupCredentials yes

Save and exit.

On VM2 (service.kerberos.com)

- T3.** The Kerberized service that you will use here is SSH. You need to configure SSH to perform a set of authentication methods including password (by default) and Kerberos.

Let's first install the following packages openssh-server and krb5-config by executing:

sudo apt install openssh-server krb5-config

You should be presented with the same questions you were asked during the krb5-kdc server installation. The answers are the same: KERBEROS.COM, kdcserver.kerberos.com and kdcserver.kerberos.com.

Check that you have /etc/krb5.conf. Edit /etc/krb5.conf to be identical to what you have for the kdcserver and client. Make sure that you have connectivity from VM2 to VM1 and VM3.

Let's configure SSH service to use Kerberos as one of the authentication methods. Uncomment the following lines in the /etc/ssh/sshd_config file. Also, change the "no" in the first line to "yes" (that is enabling Kerberos Authentication).

```
GSSAPIAuthentication yes  
GSSAPICleanupCredentials yes
```

You also need to change, in the /etc/ssh/sshd_config file, the line

```
PermitRootLogin prohibit-password
```

into

```
PermitRootLogin yes  
PasswordAuthentication yes # modern implementations require it to be explicitly enabled
```

save and exit.

Restart the SSH server for the changes to take effect

```
sudo systemctl restart sshd.service
```

Before you conclude the configuration on VM2, you need to create a password for the root account as the VMs are created without one. Execute the following commands:

```
sudo su  
passwd root
```

Enter the password helloroot (for convenience) and you are done with the configuration of VM2 (service.kerberos.com). This will be needed in the next task for copying a file from VM3 to VM2.

On VM3 (kdcserver.kerberos.com)

- T4.** You are almost there! You need to configure the kdc server (kdcserver.kerberos.com, VM3) to work with the SSH service on service.kerberos.com (VM2).

First, let's create what is known as a keytab file. Keytab is a file extracted from the kdc principal database and contains the encryption key for a service (such as SSH on service.kerberos.com). Execute the following command:

```
sudo kadmin.local
```

At the kadmin.local prompt execute the following:

```
kadmin.local: addprinc -randkey host/service.kerberos.com  
kadmin.local: ktadd -k /tmp/service.kerberos.com.keytab host/service.kerberos.com
```

Note that the ktadd command creates a random key for a principal or set of principals in the kdc database, and returns the key so that it can be saved into a keytab file. Exit the kadmin.local by entering q (for quit) and hit return.

- Q2 –

- (a) Find a way to display the content of the key.tab file in a human readable form or format?
- (b) Display the content of the key.tab file in a human readable format.
Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources

Now, copy the /tmp/service.kerberos.com.keytab file from kdcserver.kerberos.com VM3 to the service.kerberos.com server (VM2) using the following LONG one line command

```
sudo scp /tmp/service.kerberos.com.keytab \  
root@service.kerberos.com:/etc/krb5.keytab
```

On VM2, make sure that the krb5.keytab is there.

The moment of truth!

Testing SSH service with Kerberos

- T5.** On the HOST, ssh X forwarding to VM1. Launch Wireshark, configure to capture packets on ens33 and apply Kerberos as a display filter.

On VM1, log out and log back in. Destroy any lingering Ticket Granting tickets by executing:
sudo kdestroy

Always verify using sudo klist.

Request a new TGT for the root user on VM1 (from VM3) by executing:
sudo kinit root

Always verify using sudo klist. Note that this represent the initial login by the root user.

Then type the command given below (on VM1) to get access to the SSH on service.kerberos.com. The expectation is that the root user is able to login into the SSH without entering a password.

Make sure to setup your environment so that you can capture the ssh debug message. Let's try it:

```
sudo ssh -v root@service.kerberos.com
```

Note that for the first time accessing the ssh server, you would need to answer the first question with yes.

Save the debug messages into **SSO-root-VM1-SSH-VM2-WITH-TGT-TGS-E2S3T5-on-VM1.txt**

Q3 – From the debug messages generated (by selecting the –v option) explain the high-level steps that led the SSH server to GSSAPI (Kerberos related) authentication over the password method. *Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources*

If it works, then my congratulations are in order. You have experienced your first true SSO (single sign on). You are logged into the SSH server without having to enter a password. Imagine, that all services in the realm KERBEROS.COM are Kerberized, then you would be able to log into each

of them without ever having to type a password expect when you log into your machine (VM1 (client)) the first time. Terminate the connection.

Stop Wireshark on the HOST/VM1 and export two sets of files:

- (1) Apply `kerberos & (ip.addr==<client VM1 ip address> && ip.addr==<kdcserver VM3 ip address>)` as a display filter. **Export the packets into SSO-root-VM1-KDC-VM3-WITH-TGT-TGS-E2S3T5-on-X-forwarding-VM1.pcapng**
- (2) Apply `ip.addr==<client VM1 ip address> && ip.addr==<service VM2 ip address>)` as a display filter. **Export the packets into SSO-root-VM1-SSH-VM2-WITH-TGS-E2S3T5-on-X-forwarding-VM1.pcapng**

- Q4–
 - (a) why do you have 6 kerberos packets? Pay attention to the last 2
 - (b) describe the packet exchange between VM1 and VM2 Are there any Kerberos? Explain why. Make sure to use a screenshot of the relevant packets captured by Wireshark!

Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources

- T6.** Before you take the next step, let's check the tickets that you have now on VM1 by executing:
`sudo klist`

You should get two tickets for the user root: (1) the original TGT and (2) a new one (TGS) for the service you have successfully requested and granted access to through the Kerberos authentication scheme!

Let's check that this experience is real by destroying the TGT and TGS:
`sudo kdestroy`

Try to log into the SSH server again. **Make sure to setup your environment so that you can capture the ssh debug message.**
`sudo ssh -v root@service.kerberos.com`

If everything works as expected, the ssh server will ask for a password. Save the debug messages into **SSO-root-VM1-SSH-VM2-NO-TGT-NO-TGS-E2S3T6-on-VM1.txt**

Q5 – Compare the ssh debug messages in this case with the ones you explained in the answer to Q3. *Provide evidence. Reference the evidence with specific description of the relevant information. Reference your evidence. Provide citations to external sources*

Try out checkout principal

Try out another user that you have already populated in the kdcserver Kerberos database, that is, checkout. First, you have to create a TGT for checkout using
`sudo kinit checkout` # equivalent to the first and only login

Verify using
`sudo klist`

Next, execute the following command:

```
sudo ssh -v checkout@service.kerberos.com
```

It should work. Note that you do not need to enter yes. Explain why?

5 References:

[1] KDC_ERR_PREAUTH_REQUIRED vs. KDC_ERR_PREAUTH_FAILED

https://blogs.technet.microsoft.com/makeiteasy/2013/01/14/kdc_err_preauth_required-vs-kdc_err_preauth_failed/

[2] How to Setup Kerberos Server and Client on Ubuntu 20.04, <https://www.atlantic.net/dedicated-server-hosting/how-to-setup-kerberos-server-and-client-on-ubuntu-20-04/>

[3] How to Display the Keylist (Principals) in a Keytab File, <https://docs.oracle.com/cd/E19683-01/806-4078/6jd6cjs1q/index.html>

Appendix A: Static IP address using netplan

- Edit /etc/netplan/00-installer-config.yaml

```
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: no
      addresses: [192.168.1.10/24]
      gateway4: 192.168.1.2
      nameservers:
        addresses: [192.168.1.30,192.168.1.2,8.8.8.8]
```
- Execute **sudo netplan apply**
- Exit and log back in

Appendix B: Local DNS Server Configuration files

```
checkout@kdcserver:~$ sudo vim /etc/bind/named.conf.local
```

```
//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";
zone "kerberos.com" IN {
    type master;
    file "/etc/bind/kerberos.com.zone";
};
zone "1.168.192.in-addr.arpa" IN {
    type master;
```

```
file "/etc/bind/kerberos.com.rev";
};
```

checkout@kdcserver:~\$ sudo cp /etc/bind/db.local /etc/bind/kerberos.com.zone
checkout@kdcserver:~\$ sudo vim /etc/bind/kerberos.com.zone #edit to look like the following:

```
;
; BIND data file for local loopback interface
;
$TTL      604800
@         IN      SOA    kdcserver.kerberos.com. root.kdcserver.kerberos.com. (
                        2      ; Serial
                        604800 ; Refresh
                        86400  ; Retry
                        2419200 ; Expire
                        604800) ; Negative Cache TTL

@         IN      NS     kdcserver.kerberos.com.
kdcserver IN      A      192.168.1.30
client    IN      A      192.168.1.10
service   IN      A      192.168.1.20
```

checkout@kdcserver:~\$ sudo cp /etc/bind/db.127 /etc/bind/kerberos.com.rev
checkout@kdcserver:~\$ sudo vim /etc/bind/kerberos.com.rev #edit to look like the following:

```
;
; BIND data file for local loopback interface
;
$TTL      604800
@         IN      SOA    kdcserver.kerberos.com. root.kdcserver.kerberos.com. (
                        2      ; Serial
                        604800 ; Refresh
                        86400  ; Retry
                        2419200 ; Expire
                        604800) ; Negative Cache TTL

@         IN      NS     kdcserver.kerberos.com.
30        IN      PTR    kdcserver.kerberos.com.
10        IN      PTR    client.kerberos.com.
20        IN      PTR    service.kerberos.com.
```

Appendix C: Local DNS testing

checkout@kdcserver:~\$ nslookup client.kerberos.com

```
Server:      192.168.1.30
Address:     192.168.1.30#53
```

```
Name:        client.kerberos.com
Address:     192.168.1.10
```

checkout@kdcserver:~\$ nslookup service.kerberos.com

Server: 192.168.1.30
Address: 192.168.1.30#53

Name: service.kerberos.com
Address: 192.168.1.20

checkout@kdcserver:~\$ nslookup 192.168.1.10

10.1.168.192.in-addr.arpa name = client.kerberos.com.

checkout@kdcserver:~\$ nslookup 192.168.1.20

20.1.168.192.in-addr.arpa name = service.kerberos.com.

checkout@kdcserver:~\$ nslookup 192.168.1.30

30.1.168.192.in-addr.arpa name = kdcserver.kerberos.com.

checkout@kdcserver: \$ host kdcserver.kerberos.com

kdcserver.kerberos.com has address 192.168.1.30

checkout@kdcserver: \$ host client.kerberos.com

client.kerberos.com has address 192.168.1.10

checkout@kdcserver: \$ host service.kerberos.com

service.kerberos.com has address 192.168.1.20